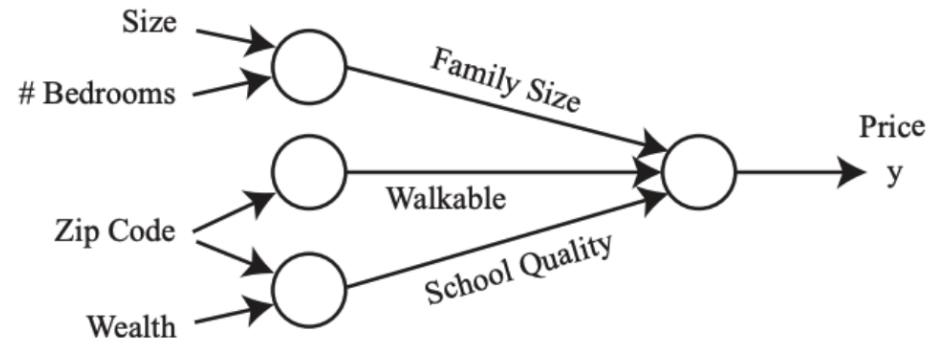


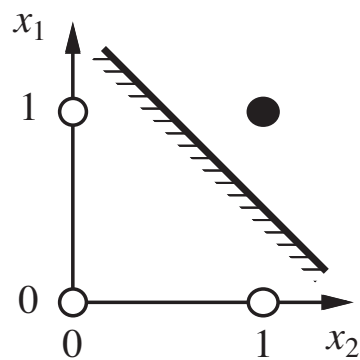


Hyperparameters to Tune:

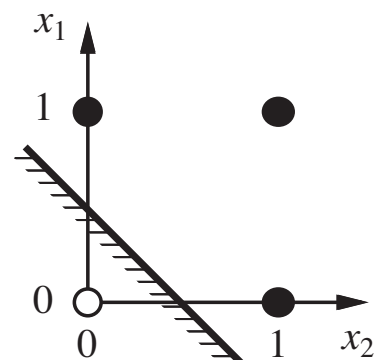
1. The number of layers
2. The number of hidden units per layer
3. The gradient descent learning rate η
4. The initialization
5. The stopping iteration or weight regularization

Not with a perceptron ☹️

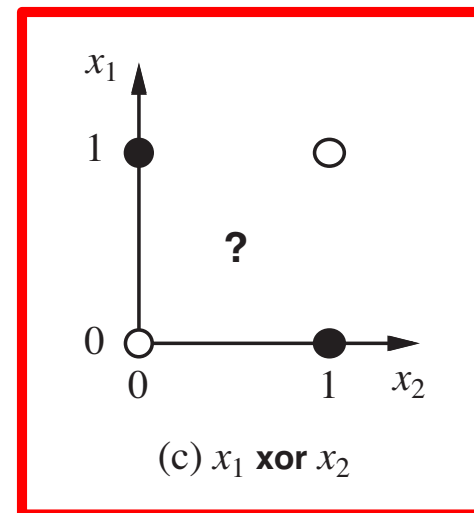
Consider Boolean operators (and, or, xor)
with four possible inputs: 00 01 10 11



(a) x_1 and x_2

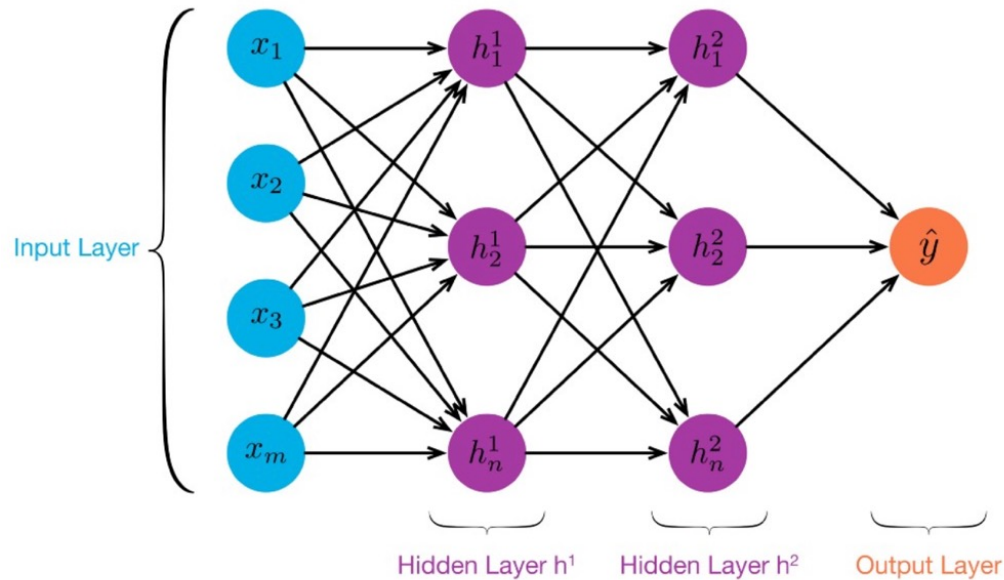


(b) x_1 or x_2



(c) x_1 xor x_2

Training examples are **not linearly separable**
for one case: $sum=1$ iff x_1 xor x_2

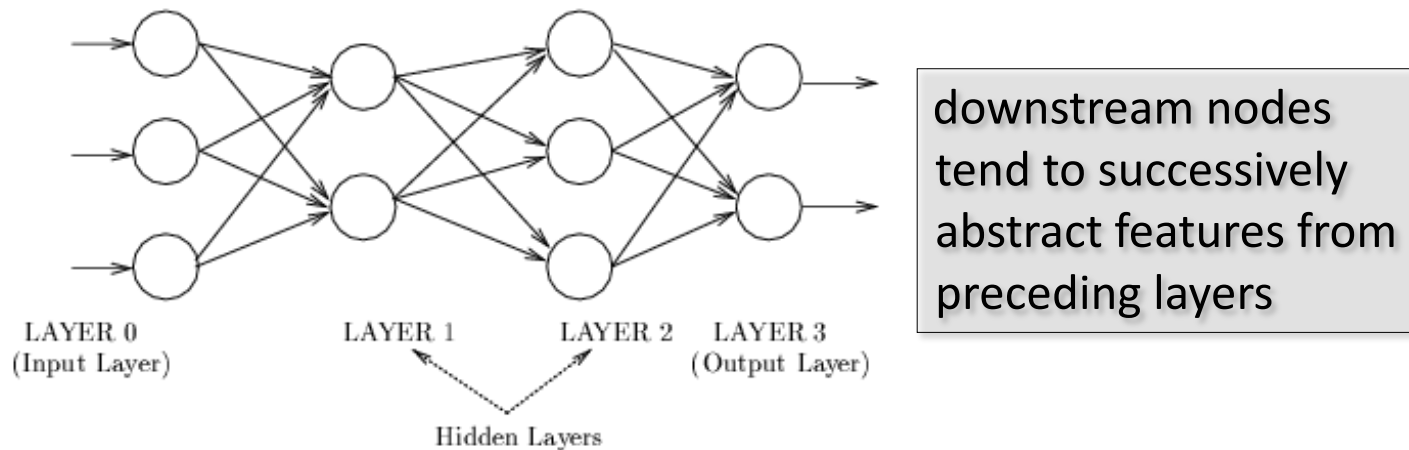


MLP: Multilayer Perceptron

- ≥ 1 “hidden layers” between inputs & output
- Can compute **non-linear functions** (why?)
- Training: adjust weights slightly to reduce error between output $\mathbf{y}$ and target value $\mathbf{t}$; repeat
- Introduced in 1980s, still used today

Feedforward Neural Network

- Connections allowed from a node in layer i only to nodes in layer $i+1$, i.e., no cycles or loops
- Simple, widely used architecture, provides a good baseline
- Backpropagation used while training, of course



Tinker With a **Neural Network** Right Here in Your Browser.
Don't Worry, You Can't Break It. We Promise.

1. Select dataset 2. Choose features 3. Add layers 4. Parameters 5. Task

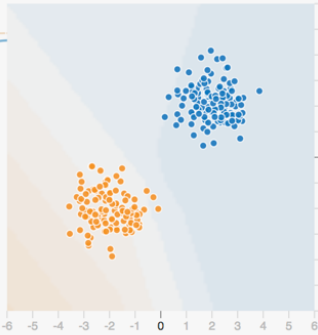
Epoch: 000,000 Learning rate: 0.03 Activation: ReLU Regularization: None Regularization rate: 0 Problem type: Classification

DATA
Which dataset do you want to use?

Ratio of training to test data: 50%
Noise: 0
Batch size: 10
REGENERATE

FEATURES
Which properties do you want to feed in?
 X_1
 X_2
 X_1^2
 X_2^2
 $X_1 X_2$
 $\sin(X_1)$
 $\sin(X_2)$

2 HIDDEN LAYERS
4 neurons 2 neurons
The outputs are mixed with varying **weights**, shown by the thickness of the lines.
This is the output from one **neuron**. Hover to see it larger.

OUTPUT
Test loss 0.435
Training loss 0.432

Colors shows data, neuron and weight values.
☐ Show test data ☐ Discretize output

[HTTP://PLAYGROUND.TENSORFLOW.ORG/](http://playground.tensorflow.org/)

Deep Learning Frameworks (1)

- Popular open-source deep learning frameworks use Python at top-level; C++ in backend
 - [TensorFlow](#) (via Google)
 - [Keras](#) (Open Source, now TensorFlow's I/F)
 - [PyTorch](#) (via Facebook)
 - [MxNet](#) (Apache)
 - [Caffe](#) (Berkeley)
- TensorFlow and PyTorch now dominate; both make it easy to specify a complicated network
- Keras is now part of TensorFlow